

Operations Assistant: a multi-agent crew on an MCP server you build

Build a small MCP server that exposes tools over local data, then build a CrewAI crew that connects to it and finishes one real task end to end, with traces, tests, and answers that cite their evidence.

BUILD	One MCP server with a few tools + a CrewAI crew that uses them to answer a question and write a sourced report.
SKILLS	MCP servers (tools, resources, transports), CrewAI (agents, tasks, crews), and the practical know-hows that make agents safe and reliable.
OUTPUT	A public repo that runs from a fresh clone, an inspectable MCP server, a working crew, saved run traces, and a demo recording.
RULE	Core carries 80, stretch 20. A notebook is not a submission. Keep it laptop-runnable with a free or local model.

1 · The scenario

A mid-sized operations team keeps its knowledge in two places: a folder of short documents such as policies, product notes, and past support tickets, and one small spreadsheet of orders or inventory. Today, staff answer routine questions by hand. They open files, search, cross-check the spreadsheet, and write a short summary.

You will build a small assistant that does this the way a real team would. A crew of agents shares one set of tools, and those tools live behind an MCP server that you build. Given a question, the crew searches the documents, reads the relevant records, and writes a short report that says where each fact came from.

2 · What you are building

Part	What it does
MCP server	Exposes 2 to 3 tools over your local data: <code>search_documents(query)</code> , <code>read_record(id)</code> , and (for Core) <code>save_report(title, content)</code> . Optionally a resource that lists the documents.
Tools	Plain Python functions with validated inputs and clear error messages, built with the official mcp SDK (FastMCP).
CrewAI crew	Two or three agents with clear roles, for example Researcher and Writer, that connect to your server through MCPServerAdapter and use its tools.
One task	Answer a business question by retrieving evidence and producing a short, sourced report.
Evidence	Saved traces of each agent step and tool call, example questions with their outputs, and tests.

The point of the week is the wiring: one server, one crew, tools shared between them, and an answer you can trust because every claim points to a document or a record.

3 · Inputs and scope rules

- Data.** A tiny corpus: about 8 to 15 short text documents (public, synthetic, or written by you) plus one small CSV of 10 to 50 rows. Keep it small enough to read by hand. This is not a data-heavy week.
- Model.** Use a local model through Ollama, or any model you already have access to. Do not require a paid API. Do not hardcode keys; use an `.env.example` file.
- Safety.** Commit only small samples. No secrets, no private data, and no personal information in documents, screenshots, or the demo.

4 · The work

MVP

build these first; the runnable minimum (~70% of core)

CORE

hardens it toward 80

Core · MVP (a runnable minimum)	Tag
Repo and setup. README, a clear folder structure, an env file, and run commands. It must run from a fresh clone.	MVP
Sample data. Commit the small document folder and the CSV, with a short note on what they contain.	MVP
MCP server with two tools. search_documents and read_record, built with the official mcp SDK. The server runs and opens in the MCP Inspector.	MVP
A crew that uses the server. Two CrewAI agents connect through MCPServerAdapter over stdio and complete the task, producing an answer.	MVP
Grounding. Every answer names the document or record each fact came from. If a tool returns nothing, the agent says so instead of inventing an answer.	MVP
A basic check. Three example questions with saved outputs, and proof from the run log that the tools were actually called, not guessed.	MVP

Core · hardening	Tag
A third tool and roles. Add save_report (writes a markdown report to an output folder) and a third agent, or a resource that lists documents. Give each agent a clear role, goal, and task.	CORE
Input validation and errors. Validate tool inputs with strict schemas and return clear messages on bad input.	CORE
Loop control and tracing. Set max_iter so an agent cannot loop forever. Turn on verbose mode and save a trace of each agent step and tool call to a file.	CORE
Tests. Unit tests that call the MCP tools directly, and one end-to-end test that runs the crew on a fixed question.	CORE
Decision log. A short log of what you tried, what you chose, and what you rejected. Your demo lives in the 5-minute clip in section 10.	CORE

Stretch (any 4, after Core runs, scored on depth)	
Two MCP servers	Add a second server (for example a public filesystem or fetch server) and give different tools to different agents.
Human approval gate	Require human approval before save_report writes anything, or before any action that changes data.
A self-check step	Add an agent or step that checks the report's claims against the retrieved evidence and flags anything unsupported.
Planner and executor	Use CrewAI's hierarchical process: a manager agent plans, worker agents execute.
Observability	Save a structured trace of every tool call with timings and token counts, and produce a small run report.
Remote transport	Run the server as a separate process over SSE or streamable HTTP instead of stdio.
Prompt-injection test	Hide a malicious instruction inside one document and show your guardrails refuse it. This is the real security lesson of the week.

Stretch is scored on depth, not count: up to 5 marks per task, capped at 20. Two strong tasks beat four shallow ones.

5 · Practical know-hows to apply

These are the lessons that separate a demo from a system. Build them in.

- **Treat every tool input as untrusted.** Inputs reach your tools through an LLM, not directly from a person. Validate them with strict schemas, and never pass raw input into shell commands or file paths without checking.
- **A stdio MCP server runs code on your machine.** Only connect to servers you trust. If you use a public server, read what it does first.
- **Watch for the rug pull.** A server can change its tool definitions after you approve them. Pin and review the tools you depend on.
- **Stop runaway agents.** Set `max_iter` and a step limit. An agent without a ceiling can loop and burn time or tokens.
- **Close the connection.** With `MCPServerAdapter`, use the context manager (with ... as tools:) or call `stop()` in a finally block.
- **Stay free and reproducible.** Use a local model, keep keys in environment variables, and ship a small sample so the project runs from a clone.

6 · How to run it (recommended path)

Everything here runs locally and free

- ✓ Python 3.11 or 3.12, managed with `uv`.
- ✓ MCP server: install the official SDK with `pip install mcp` (it includes `FastMCP`). Test with the MCP Inspector.
- ✓ Crew: `pip install crewai 'crewai-tools[mcp]'`. Connect over stdio with `StdioServerParameters`.
- ✓ Model: a local model through Ollama, or any model you have. No paid API required.

7 · How it is scored

Area	Marks	What earns the marks
MVP tasks	55	Repo, sample data, a working MCP server with two tools, a crew that uses them, grounded answers, and it runs from a clone
Core hardening	15	A third tool and roles, input validation, loop control and tracing, tests, and a demo
AI-assisted engineering	10	A decision log, an AI red-team review of your own server, and verified fixes
Stretch	20	Advanced features that work, marked on depth (any four, five each)
Total	100	

Core work caps at 80. Stretch is the only way past 80. Marks reward a system that runs and answers you can trust, not a clever demo that cannot be explained.

8 · Minimum acceptance

- The repo runs from a fresh clone using the documented commands.
- The MCP server runs and can be opened in the MCP Inspector.
- The crew connects to the server and completes the task on sample data.
- Every answer cites the document or record behind each fact, and refuses when no evidence is found.
- Your 5-minute clip is submitted and covers the points in section 10.
- No secrets, keys, or private data are committed.

9 · Submit and reflect

Submit: the GitHub repo, a README with setup commands, the server and crew code, sample data, three example questions with saved outputs, run traces, tests, your 5-minute clip (section 10), a decision log, an AI usage log, and a short reflection.

In `reflection.md`, answer:

- Why these tools and these agent roles, over the alternatives you considered?
- What broke first when you connected the crew to the server, and what did you change?
- Show one answer the crew got wrong or ungrounded. How did your guardrail catch it, or why did it slip through?
- Where is the biggest security risk in your server, and how did you reduce it?
- What would you change before letting this touch real company data?

How this is checked

Your 5-minute clip (section 10) and a short live walkthrough within 48 hours of the deadline are how we confirm the work is yours. Any function in your server or crew may be picked at random for you to explain. AI-assisted code is fine and expected. Code you cannot explain does not count.

10 · Your 5-minute clip

The last task is a short clip where you talk through your own work. It does two things at once: it shows the project is yours, and it is your first rehearsal of how you will talk about this in an interview. Record about five minutes, add the file to your repo (a demo folder or a release) or upload it as an unlisted link, and submit the link with your solution.

One continuous take is fine and you do not need editing. Keep your face or voice on, and share your screen for the demo part.

Time	What to cover (about five minutes)
~1 min	The pitch. What you built and why it matters, the way you would open in an interview.
~1 min	Show it run. Ask one question, show the crew calling your MCP tools, and show the answer with its sources.
~1.5 min	One decision and one failure. A real choice you made and why, and one thing that broke and how you fixed it.
~1 min	What you learned. Explain in your own words what MCP and CrewAI do, the one security risk you took seriously, and the two or three sources you used most.
~30 sec	What is next. What you would change before this touched real company data.

If you can do this clearly in five minutes, you can do it in an interview. That is the real goal.

11 · References and videos to follow

Read the docs first, then follow the videos to build each part.

Documentation

What MCP is: <https://modelcontextprotocol.io/docs/getting-started/intro>

MCP Python SDK (FastMCP): <https://github.com/modelcontextprotocol/python-sdk>

Build an MCP server, step by step: <https://gofastmcp.com/tutorials/create-mcp-server>

MCP Inspector (test your server): <https://github.com/modelcontextprotocol/inspector>

CrewAI docs: <https://docs.crewai.com>

CrewAI + MCP (includes an official video walkthrough): <https://docs.crewai.com/en/mcp/overview>

CrewAI tools (the MCP adapter): <https://github.com/crewAIInc/crewAI-tools>

Anthropic MCP launch post (background): <https://www.anthropic.com/news/model-context-protocol>

Videos to follow and replicate

Build an MCP server with FastMCP: https://www.youtube.com/watch?v=_mUuhOwv9PY

Build a CrewAI multi-agent crew: <https://www.youtube.com/watch?v=K2UAE1OIC8s>

A worked CrewAI + MCP server example: <https://www.youtube.com/watch?v=f05kjsjqdsE>

For the integration step, the official walkthrough embedded on the CrewAI MCP page above is the most reliable reference, since it is maintained with the library.